Final TOC Project Proposal

The Halting Problem: Can a Computer Predict Whether a Program Will Stop?

Aligned with Module 5

Theory of Computation

Software-based project

1. Project overview

The Halting Problem is a fundamental problem in Theory of Computation that investigates whether it is possible to determine if a given Turing Machine will eventually halt or continue running indefinitely for a particular input.

Our project aims to develop an interactive Halting Problem Simulator that demonstrates Turing Machine execution, language acceptance, Universal Turing Machine simulation and the undecidability of the Halting Problem.

The application will allow users to create or select Turing Machines, provide inputs, observe their execution step by step and examine their state transitions and tape configurations. It will also demonstrate why a universal algorithm capable of correctly predicting the halting behaviour of every possible Turing Machine cannot exist.

The central focus will be to visually demonstrate the theoretical concepts and mathematical reasoning behind Alan Turing's proof of the undecidability of the Halting Problem.

2. Project objectives

1. Implement a simulator for Turing Machines and demonstrate their execution.
2. Visualize instantaneous descriptions, tape operations, head movements and state transitions.
3. Demonstrate language acceptance using Turing Machines.
4. Explain the concept and working of a Universal Turing Machine.
5. Demonstrate the difference between recursive and recursively enumerable languages.

6. Illustrate the Halting Problem and its undecidability through a self-referential construction and proof by contradiction.

3. What exactly will the application do?

The application will consist of five interconnected modules.

Module 1 – Turing Machine Simulator

Turing Maschine Simulator – ABZ – Ausbildungs- und Beratungszentrum für Informatikunterricht der ETH Zürich

abz.inf.ethz.ch

This is the foundation of the application.

Users will be able to select predefined Turing Machines or create simple machines by defining their states and transition rules.

Features:

- Display the Turing Machine's tape.
- Show the current state.
- Visualize the read/write head.
- Execute one transition at a time.
- Show the symbol being read and written.
- Display left and right head movements.
- Identify accepting and rejecting states.

TOC concepts: Turing Machine definition, representation, instantaneous description and language acceptance.

Module 2 – Turing Machine Execution and Language Acceptance

Department of Computer Science and Technology – Raspberry Pi: Introduction: What is a Turing machine?

cl.cam.ac.uk

Users provide an input string and observe how a Turing Machine processes it.

Features:

- Input string entry.
- State transition visualization.
- Step-by-step execution.

- Execution history.
- Final result: Accepted, Rejected or execution still continuing.

The application will demonstrate how a Turing Machine accepts or rejects strings belonging to a language.

TOC concepts: Language acceptance by Turing Machines and instantaneous descriptions.

Module 3 – Universal Turing Machine

Automata Theory Tutorial

tutorialspoint.com

This module demonstrates how a Universal Turing Machine can simulate another Turing Machine.

Features:

- Select a Turing Machine.
- Provide its encoded description and input.
- Simulate its execution through a universal simulator.
- Observe the simulated machine's states and tape operations.

The objective is to demonstrate that one general-purpose computational machine can simulate different Turing Machines.

TOC concepts: Universal Turing Machine and variants of Turing Machines.

Module 4 – Recursive and Recursively Enumerable Languages

PPT - Undecidability PowerPoint Presentation, free download - ID:780120

slideserve.com

This module explains the difference between two important classes of languages.

Features:

- Demonstrate a Turing Machine that always halts and accepts or rejects.
- Demonstrate a recognizer that accepts valid inputs but may continue indefinitely for invalid inputs.
- Compare their execution behaviours.
- Display the differences in a comparison table.

TOC concepts: Recursive languages and recursively enumerable languages.

Module 5 – Halting Problem and Undecidability

From Self-Referential Paradoxes to Intelligence | by CP Lu, PhD | Medium

cplu.medium.com

This is the main module and the central focus of the project.

Users will explore whether an algorithm can predict if an arbitrary Turing Machine will halt for a given input.

Features:

- Select different Turing Machines.
- Observe terminating and non-terminating execution examples.
- Demonstrate a hypothetical halting detector.
- Introduce a specially constructed machine that behaves opposite to the detector's prediction when run on its own description.
- Visualize the resulting contradiction.

The application will explain why a universal halting detector cannot exist.

TOC concepts: Halting Problem, undecidability and undecidable problems about Turing Machines.

4. Additional syllabus concepts to include

These will be included as a separate Theory Explorer section rather than being forced into the main simulator.

Syllabus concept	How it will be demonstrated
Church's Thesis	Explanation of the relationship between algorithms and Turing Machine computation.
Countability	Explanation of countable sets and their relevance to computation.

Variants of Turing Machines	Comparison of different Turing Machine models.
Rice's Theorem	Explanation of a theorem concerning undecidable properties of Turing Machines.
Post Correspondence Problem (PCP)	A small interactive example showing how the correspondence problem works and its undecidability.
Undecidable problems about TMs	Examples of problems that cannot be decided by a Turing Machine.

These are educational sections, not prerequisites for implementing the main Halting Problem demonstration.

5. Complete build plan

I recommend Python as your primary programming language, with HTML, CSS and JavaScript for the interface.

Proposed technology stack

Component	Technology
Frontend	HTML, CSS, JavaScript
Backend	Python
Web framework	Flask

Turing Machine engine	Custom Python implementation
State diagrams	Graphviz or Mermaid
Tape visualization	HTML/CSS/JavaScript
Storage	JSON files for predefined machines

Why this stack?

- Python makes it straightforward to implement Turing Machine transitions.
- Flask connects the simulator to the user interface.
- JavaScript provides interactive tape and state visualizations.
- JSON makes it easy to store machine definitions and transition rules.

Phase 1 – Develop the Turing Machine engine

Highest priority

First, build the actual computational logic in Python, independently of the website.

Implement:

1. Turing Machine definition.
2. Tape representation.
3. Read/write head.
4. State transition function.
5. Left and right movements.
6. Accept and reject states.
7. Step-by-step execution.

A transition can be represented as:

$$\delta(q_0, 0) = (q_1, 1, R)$$

Meaning:

- Current state: q_0
- Read symbol: 0
- Next state: q_1
- Write symbol: 1
- Move head: Right

Deliverable: A working Python Turing Machine simulator.

Phase 2 – Build the interactive frontend

Create the website interface.

Implement:

- Home/dashboard page.
- Turing Machine selection.
- Input string field.
- Tape visualization.
- Current state display.
- Step, Run and Reset controls.
- Execution history.

Deliverable: A user can interact with a Turing Machine and see its execution.

Phase 3 – Implement Universal Turing Machine simulation

Build the capability to simulate different encoded Turing Machines through a common simulation engine.

Implement:

- Machine description encoding.
- Input encoding.
- Universal simulation logic.
- Execution visualization.

Deliverable: Demonstrate how one simulator can simulate different Turing Machines.

Phase 4 – Implement recursive and recursively enumerable language demonstrations

Build two examples:

- A decider that always halts.
- A recognizer that may not halt for inputs outside its language.

Show their different execution behaviours.

Deliverable: An interactive comparison of the two language classes.

Phase 5 – Implement the Halting Problem demonstration

This is your most important development phase.

Build the hypothetical halting detector explanation and the self-referential contradiction.

The demonstration should follow this sequence:

Hypothetical HALT(P, I)

Receives a machine and its input

Construct a machine that behaves opposite to the prediction

Run the constructed machine on its own description

Predicts HALT

Constructed machine loops forever.

Predicts LOOP

Constructed machine halts.

Contradiction

Therefore, the assumed universal halting detector cannot exist.

Deliverable: An interactive demonstration of the undecidability proof.

Phase 6 – Complete the Theory Explorer

Add the remaining Module 5 concepts:

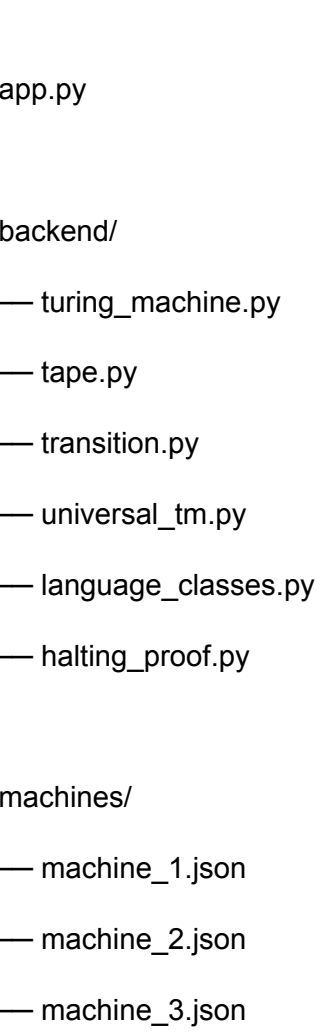
- Church's Thesis.
- Countability.
- Variants of Turing Machines.
- Rice's Theorem.
- PCP.
- Other undecidable problems about Turing Machines.

Deliverable: A complete educational section covering the remaining syllabus topics.

6. Suggested project folder structure

Keep the code modular so each TOC concept has its own implementation.

halting-problem-visualizer/



```
│
├── templates/
│   ├── index.html
│   ├── simulator.html
│   ├── universal_tm.html
│   ├── language_classes.html
│   ├── halting_problem.html
│   └── theory.html
│
├── static/
│   ├── css/
│   ├── js/
│   └── images/
│
└── README.md
```

7. Suggested development timeline

Assuming you have approximately two weeks:

Days	Development task
1–2	Understand and implement Turing Machine logic
3–4	Tape, transitions and instantaneous descriptions

5–6	Frontend and interactive simulator
7–8	Universal Turing Machine simulation
9	Recursive and RE language demonstrations
10–11	Halting Problem and contradiction demonstration
12	Remaining Module 5 theory sections
13	Testing and debugging
14	Final presentation and documentation

8. Important implementation considerations

There are three things you should be careful about:

1. Do not claim that your application solves the Halting Problem.

It demonstrates the problem and its undecidability. The universal halting detector is hypothetical and cannot actually be implemented.

2. Do not confuse execution limits with non-termination.

If a machine exceeds 1,000 execution steps, your application should report that the execution limit was reached, not that the machine definitely never halts.

3. Keep the Turing Machine simulator mathematically accurate.

The tape should be expandable, the transition rules should be followed correctly, and accepting/rejecting states should be distinguished from a machine that simply continues running.

9. Final project proposal for teacher submission

The Halting Problem: Can a Computer Predict Whether a Program Will Stop?

1. Introduction

The Halting Problem is a fundamental problem in Theory of Computation that deals with determining whether a given Turing Machine will eventually halt or continue executing indefinitely for a particular input.

This project aims to develop an interactive Halting Problem Simulator that demonstrates the working of Turing Machines and explores the limitations of computation through the undecidability of the Halting Problem.

2. Objectives

- Implement and visualize Turing Machine execution.
- Demonstrate instantaneous descriptions, tape operations and state transitions.
- Explore language acceptance by Turing Machines.
- Demonstrate Universal Turing Machine simulation.
- Compare recursive and recursively enumerable languages.
- Explain the Halting Problem and its undecidability through a self-referential construction and proof by contradiction.
- Provide educational demonstrations of other concepts from Turing Machine Theory and Undecidability.

3. Proposed Modules

Module 1: Turing Machine Simulator

An interactive simulator that allows users to observe tape operations, head movements, state transitions and machine execution.

Module 2: Language Acceptance

Demonstrates how Turing Machines process input strings and accept or reject strings belonging to a language.

Module 3: Universal Turing Machine

Demonstrates how a Universal Turing Machine can simulate other Turing Machines using their encoded descriptions and inputs.

Module 4: Recursive and Recursively Enumerable Languages

Provides interactive demonstrations comparing machines that always halt with machines that may continue indefinitely for inputs they do not accept.

Module 5: Halting Problem and Undecidability

The central module, demonstrating the hypothetical halting detector, self-referential construction and contradiction that establishes the undecidability of the Halting Problem.

Module 6: Theory Explorer

Provides explanations and examples of Church's Thesis, countability, variants of Turing Machines, Rice's Theorem, PCP and other undecidable problems about Turing Machines.

4. Technology Stack

- Programming Language: Python
- Backend: Flask
- Frontend: HTML, CSS and JavaScript
- Visualization: JavaScript and Mermaid/Graphviz
- Machine Configuration: JSON

5. Expected Outcome

The expected outcome is an interactive educational application that demonstrates the working and limitations of Turing Machines. It will provide a practical visualization of important concepts from Turing Machine Theory and Undecidability, with particular emphasis on understanding why the Halting Problem cannot be solved by a universal algorithm.

6. Syllabus Alignment

The project is primarily based on Module 5 – Turing Machine Theory and Undecidability, covering:

- Turing Machines: Definition and Representation

- Instantaneous Description
- Language Acceptance by Turing Machines
- Variants of Turing Machines
- Universal Turing Machine
- Church's Thesis
- Recursive and Recursively Enumerable Languages
- Countability
- Halting Problem
- Undecidability
- Undecidable Problems about Turing Machines
- Rice's Theorem
- Post Correspondence Problem

10. Final recommendation

I would keep the project title unchanged:

The Halting Problem: Can a Computer Predict Whether a Program Will Stop?

And make the Turing Machine simulator the foundation, with the Halting Problem contradiction as the main highlight.

Your project should ultimately answer three questions:

1. How does a Turing Machine compute? — Turing Machine simulator.
2. How can one machine simulate another? — Universal Turing Machine.
3. Are there limits to what computation can decide? — Halting Problem and undecidability.

These three questions give your project a clear academic progression, and all three are directly rooted in your Module 5 syllabus.